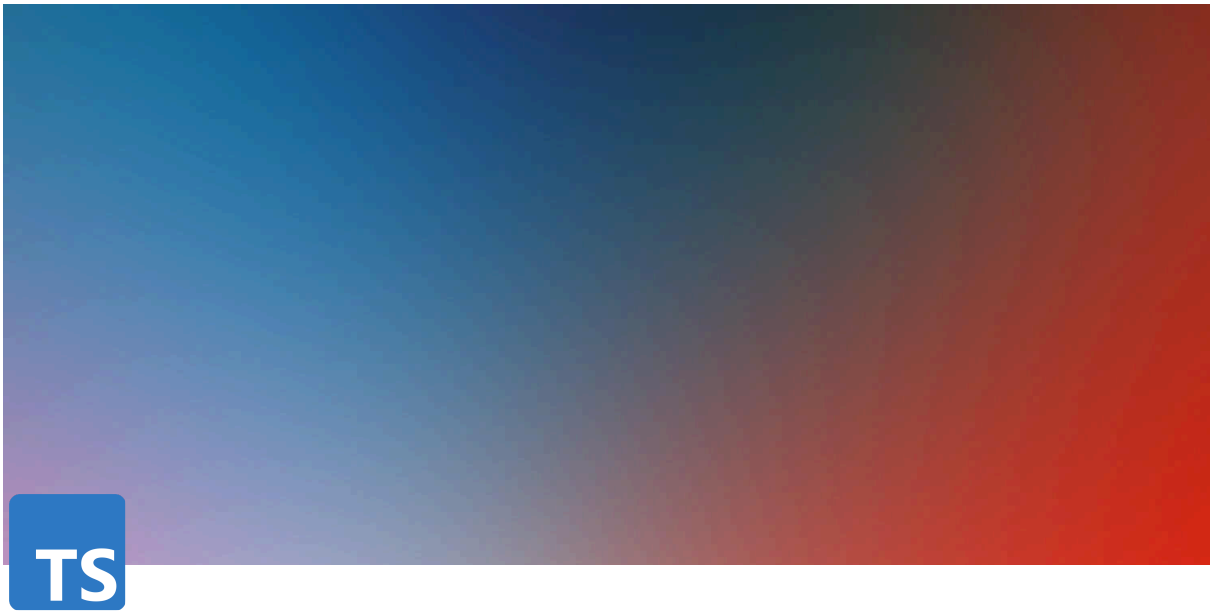

Phase 2.5

📅 Phase 2.5 — Backend with TypeScript

📌 Month 1 → TypeScript Backend Foundation + Architecture

🗓️ Week 1: TypeScript Fundamentals for Backend

1. Understand TypeScript vs JavaScript.
2. Learn:
    - Types, interfaces
    - Enums
    - Generics
    - Utility types
3. Understand strict mode & tsconfig.
4. Practice modular project structure.

Example

```
interface User {
  id: number;
  email: string;
  role: "ADMIN" | "USER";
}
```

🗓️ Week 2: Node.js + Express with TypeScript

1. Setup Node.js project with TypeScript.
2. Install:
   - express
   - @types/express
   - ts-node-dev
3. Understand:
   - Request/Response types
   - Middleware typing
4. Proper folder structure.

Clean Structure (Layered)

```
src/
 ├── config/
 ├── controllers/
 ├── services/
 ├── repositories/
 ├── models/
 ├── routes/
 ├── middlewares/
 ├── utils/
 └── app.ts
```

🗓️ Week 3: Database with TypeORM or Prisma

Choose **Prisma (recommended)**.

1. Setup PostgreSQL connection.
2. Define schema.
3. Run migrations.
4. Understand relations.

Example (Prisma schema)

```
model User {
  id    Int    @id @default(autoincrement())
  email String @unique
  role  String
}
```

🗓️ Week 4: Clean Architecture & DTO Pattern

1. Understand layered architecture:
   - Controller → Service → Repository
2. Learn DTO (Data Transfer Object).
3. Implement validation with:
   - class-validator OR zod
4. Centralized error handling.

Flow Example

```
Route → Controller → Service → Repository → DB
```

📌 Month 2 → Authentication, Authorization, Advanced Patterns

🗓️ Week 5: JWT Authentication

1. Implement register/login.
2. Hash password (bcrypt).
3. Generate JWT.
4. Protect routes with middleware.

JWT Middleware Example

```
export const authMiddleware = (
  req: Request,
  res: Response,
  next: NextFunction
) => {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ message: "Unaut
horized" });
  next();
};
```

🗓️ Week 6: Authorization & Role System

1. Role-based access (ADMIN / USER).
2. Permission middleware.
3. Resource ownership validation.
4. Handle 401 vs 403 correctly.

🗓️ Week 7: Pagination, Filtering, Sorting

1. Query parameters.
2. Offset/limit pattern.
3. Search implementation.
4. Prisma filtering.

Example:

```
const users = await prisma.user.findMany({
  skip: 10,
  take: 10,
  where: { email: { contains: "gmail" } },
});
```

🗓️ Week 8: Logging, Environment, and Error Handling

1. dotenv for environment config.
2. Winston or Pino for logging.
3. Global error handler middleware.
4. Proper HTTP error classes.

📌 Month 3 → Build "Production-Ready Project API"

🎯 Build: **Project Collaboration API (Industry-level)**

Entities:

- User
- Workspace
- Project
- Task
- TaskAssignment

🗓️ Week 9: Project Setup & Architecture

1. Setup clean architecture.
2. Design ERD.
3. Implement database schema.
4. Setup ESLint + Prettier.

5. Setup Husky (pre-commit hooks).

🗓️ Week 10: Core Features

1. Workspace CRUD.
2. Project CRUD inside workspace.
3. Task CRUD inside project.
4. Assign/unassign users.
5. Task status & priority logic.

Handle:

- Not found errors
- Invalid relations
- Business rules

🗓️ Week 11: Advanced Features

1. Pagination everywhere.
2. Filtering by:
    - status
    - assigned user
3. Search project/task name.
4. Soft delete implementation.
5. Refresh token system.

🗓️ Week 12: Testing + Deployment

1. Unit testing with:
    - Jest
2. Integration testing.
3. Mock Prisma.
4. API documentation (Swagger).

5. Dockerize application.
6. Deploy to:
   - Railway
   - Render
   - Fly.io

🏗️ Recommended Real-World Structure

```
src/
 ├── modules/
 │    ├── user/
 │    │     ├── user.controller.ts
 │    │     ├── user.service.ts
 │    │     ├── user.repository.ts
 │    │     ├── user.dto.ts
 │    │     └── user.routes.ts
 │
 ├── shared/
 │    ├── middleware/
 │    ├── errors/
 │    ├── utils/
 │
 ├── config/
 ├── app.ts
 └── server.ts
```

This is **modular + scalable structure** used in real production systems.

📚 What You Will Master After Phase 2.5

Skill	Level
TypeScript	Strong
Clean Architecture	Production
JWT + Auth	Advanced

Skill	Level
Role-based Authorization	Industry
Testing	Professional
Docker + Deployment	Real-world